

AMOUROUX Paul
HUYNH Elodie

Projet Analyse Syntaxique - S5

Introduction :

Le projet consiste à écrire un analyseur syntaxique en utilisant flex et bison.

Le langage analysé est le C, pour vérifier l'intégrité d'un programme, et ainsi regarder si celui-ci respecte bien les règles du langage.

Il a en plus la possibilité d'afficher les composantes d'un programme à analyser sous forme d'arbre (en utilisant tree.c).

Fonctionnement :

Pour lancer les tests unitaires, deux commandes sont possibles :

`make testG` pour les tests unitaires qui fonctionnent

`make testB` pour les tests unitaires qui ne fonctionnent pas

Tests unitaires :

Voici la liste exhaustive des tests unitaires pour tester notre analyseur :

Good test:

- 00: Main vide avec aucun retour
- 01: Main basique en une ligne
- 02: Main basique sur plusieurs lignes
- 03: Main basique avec des commentaires basiques
- 04: Main basique avec des commentaires sur plusieurs lignes
- 05: Main basique avec un paramètre
- 06: Main basique avec plusieurs paramètres
- 07: Main avec déclaration d'une variable
- 08: Main avec déclaration de multiples variables sur une ligne
- 09: Main avec affectation d'une variable int
- 10: Main avec affectation d'une variable char
- 11: Main avec affectation d'une expression OR
- 12: Main avec affectation d'une expression AND
- 13: Main avec affectation d'une expression EQ
- 14: Main avec affectation d'une expression ORDER <
- 15: Main avec affectation d'une expression ORDER <=
- 16: Main avec affectation d'une expression ORDER >
- 17: Main avec affectation d'une expression ORDER >=
- 18: Main avec affectation d'une expression ADDSUB unaire
- 19: Main avec affectation d'une expression ADDSUB binaire
- 20: Main avec affectation d'une expression DIVSTAR
- 21: Main avec instruction IF (simple)
- 22: Main avec instruction IF ELSE
- 23: Main avec instruction WHILE (simple)
- 24: Main avec instruction WHILE (comparaison)
- 25: Main avec instruction IF (comparaison)

- 26: Main avec instruction IF imbriqués
- 27: Fonction paramètre void
- 28: Fonction avec un paramètre
- 29: Fonction avec plusieurs paramètres
- 30: Test avec variables globales et plusieurs fonctions
- 31: Fonction avec déclaration statique
- 66: Test avec plusieurs fonctions
- 67: Test avec plusieurs fonctions
- 68: Test avec plusieurs fonctions

Syn-err test:

- 01: Uniquement un commentaire
- 02: Pas de type de retour
- 03: Pas de parenthèse fermante après un IF
- 04: Déclaration de IF en tant que variable
- 05: Pas de parenthèse fermante après un WHILE
- 06: Pas de ; après instruction
- 07: Pas d'accolade ouvrante après déclaration de fonction
- 08: Pas d'accolade fermante
- 09: Mauvaise déclaration de variable (déclaration vide)
- 10: Mauvaise utilisation IF
- 11: Mauvaise déclaration de variable (pas de nom)
- 12: Erreur syntaxe
- 13: Condition WHILE vide
- 14: Condition IF vide
- 15: Mauvais argument (parenthèse en trop)
- 16: Suite de comparaisons ORDER
- 17: Manque de virgule pour une double affectation
- 18: Déclarations non séparées par un point virgule
- 19: Déclarations avec une virgule en trop
- 20: Fonction sans corps
- 21: Déclaration de plusieurs fonctions à la suite
- 22: Mauvaise utilisation de DIVSTAR (deux à la suite)
- 23: Fonctions imbriquées l'une dans l'autre

Difficultés rencontrées :

Nous avons parfois eu du mal à trouver les erreurs de grammaire, ce qui nous empêchait d'avoir une note de 100/100 sur le site du bac à sable.

Conclusion :

En conclusion, nous avons un analyseur syntaxique qui étudie le langage C et cet analyseur permet de dessiner l'arbre lié au programme.

